BookMyShow

DevOps CI/CD Implementation Project

Deployment of an Online Ticketing & Event Management Platform

Using AWS, Docker, Kubernetes (EKS), Jenkins, Ansible, SonarQube, Trivy, Prometheus & Grafana

Project Documentation

Domain: E-Commerce & Entertainment
Repository: github.com/varunspark/Book-My-Show
Live Application: http://bookmyshow.hsmo.online
Cloud Provider: AWS (Region: ap-south-1, Mumbai)
Document prepared as a complete build log and reference guide

Table of Contents

1. Executive Summary

This document records the complete, end-to-end build of a production-style DevOps pipeline for the BookMyShow movie ticketing application. The project began with a single AWS EC2 instance and a cloned GitHub repository, and was built up step by step into a fully automated CI/CD platform running on Amazon EKS (Elastic Kubernetes Service).

The finished system covers every stage of the software delivery lifecycle described in the original project brief: source control, automated build and quality scanning, containerization, orchestration, configuration-managed deployment, monitoring, and rollback. Every component described below was actually built, tested, and verified working, including the real failures encountered along the way and how each was diagnosed and resolved, since troubleshooting real infrastructure problems is itself a core part of a DevOps engineering exercise.

1.1 What Was Delivered

Capability	Status	Tool(s) Used
Source control & webhook automation	Working	Git, GitHub Webhooks
CI/CD pipeline orchestration	Working	Jenkins
Code quality analysis	Working	SonarQube
Security vulnerability scanning	Working	Trivy (filesystem + image)
Containerization	Working	Docker (multi-stage build)
Container registry	Working	AWS ECR
Container orchestration	Working	AWS EKS (Kubernetes 1.34)
Configuration-managed deployment	Working	Ansible
Monitoring & dashboards	Working	Prometheus, Grafana
Custom domain / DNS	Working	AWS Route 53
Rollback capability	Proven working	Versioned image tags + Ansible

1.2 High-Level Pipeline Flow

The automated flow, once a developer pushes code to the main branch, is:

1. Developer pushes code to GitHub (main branch)
2. GitHub webhook instantly notifies Jenkins
3. Jenkins checks out the latest code
4. SonarQube scans the source code for quality issues
5. Trivy scans the filesystem for known vulnerabilities
6. Docker builds a production image (multi-stage: Node.js build, then nginx runtime)
7. Trivy scans the built image for OS-level vulnerabilities
8. The image is tagged with a unique build number and pushed to AWS ECR

9. An Ansible playbook deploys the new image to the EKS cluster
10. Kubernetes performs a rolling update with zero downtime
11. Prometheus and Grafana continuously monitor the live application

2. Architecture Overview

2.1 Infrastructure Components

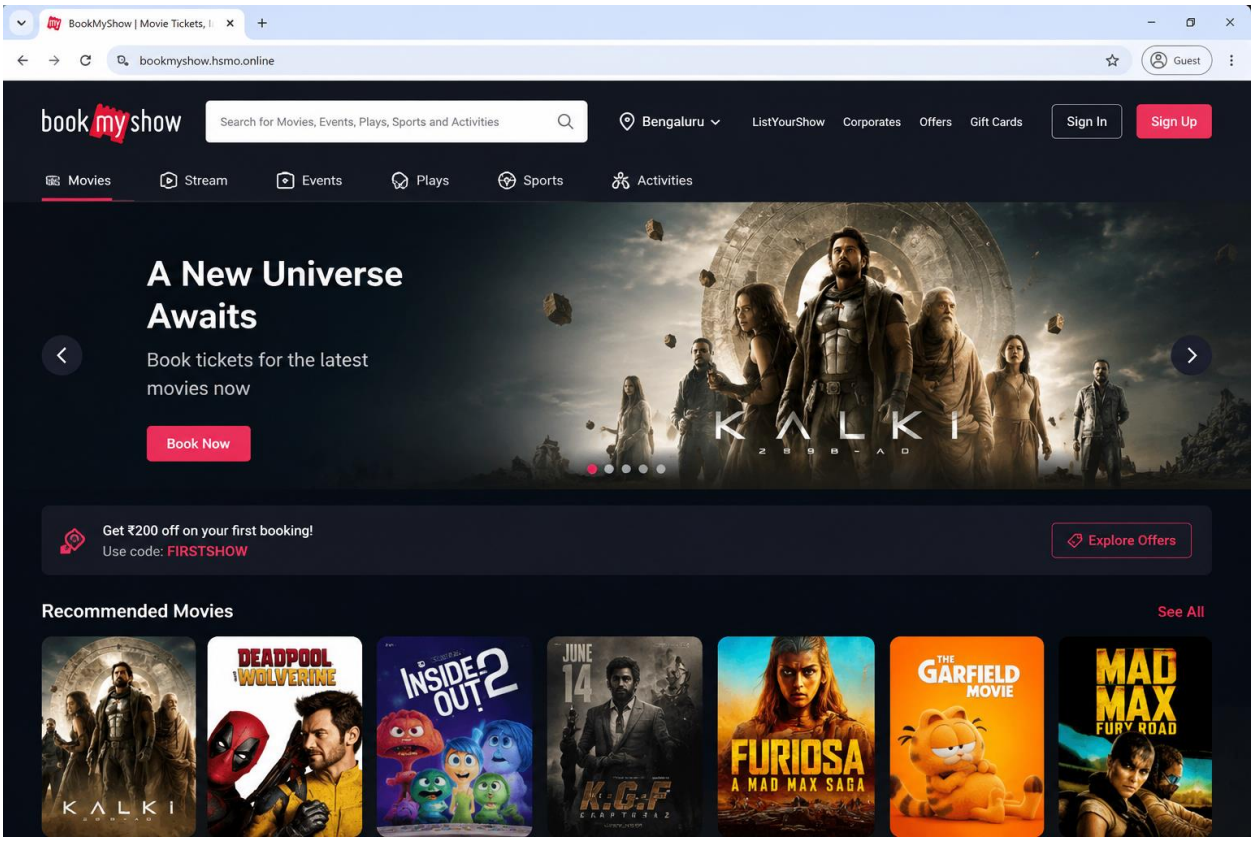
Component	Purpose	Details
EC2 Instance	Control / build host	t2.large, Ubuntu. Hosts Jenkins, SonarQube, Ansible, Docker, kubectl, AWS CLI
EKS Cluster	Application runtime	bookmyshow-cluster, 2 worker nodes (t3.medium), region ap-south-1
ECR Repository	Container image storage	154810814607.dkr.ecr.ap-south-1.amazonaws.com/bookmyshow
Route 53	DNS	hsmo.online hosted zone; bookmyshow.hsmo.online alias record
Classic Load Balancer	Traffic entry point	Auto-provisioned by the Kubernetes Service (type: LoadBalancer)
IAM Role: EC2-S3-Access-Role	Permissions for EC2 + Jenkins	Extended with ECR, EKS, and Administrator policies

2.2 Why an EC2 control box plus a separate EKS cluster?

Two different Kubernetes-related tools appear in this project, serving different purposes:

- kind (Kubernetes-in-Docker), installed early on EC2, is a local, throwaway cluster used only for quick local testing. It does not connect to AWS and was not used for the final deployed application.
- AWS EKS is the real, managed, cloud-hosted Kubernetes cluster that actually serves the live application traffic and is the target of every automated deployment.

The EC2 instance itself does not run the application. It acts purely as the automation and control plane: it hosts Jenkins (the CI/CD engine), SonarQube (code quality server), and the command-line tools (kubectl, aws, ansible, docker, trivy) used to build, scan, and deploy the application onto the separate EKS cluster.



3. Phase 1: Running the Application with Docker

Before any cloud infrastructure was touched, the application was built and run locally on the EC2 instance using Docker. This validated that the source code actually worked before adding the complexity of Kubernetes, CI/CD, or AWS services on top of it.

3.1 Environment Preparation

Connected to the EC2 instance via SSH from a Windows laptop using Git Bash:

```
ssh -i "path\to\your-key.pem" ubuntu@<ec2-public-ip>
```

Confirmed the repository (varunspark/Book-My-Show) was already cloned, and inspected its structure. The repository was found to contain:

- A root-level Dockerfile (unused by the actual pipeline, a leftover/alternative version)
- bookmyshow-app/ - the actual React application source code and the Dockerfile used by Jenkins
- k8s/ - Kubernetes manifests (Deployment, Service, HPA, PodDisruptionBudget, NetworkPolicy, ConfigMap/Secret)
- Jenkinsfile - a pre-written CI/CD pipeline definition (requiring significant rework, detailed in Phase 5)

3.2 First Build Attempt and the Chokidar Bug

The application is a Create React App (CRA) project. The first build used the existing bookmyshow-app/Dockerfile, which ran npm start (the React development server) inside the container on port 3000.

Running the container produced a real, repeatable crash:

```
Error: No version of chokidar is available. Tried chokidar@2 and chokidar@3.
Cannot find module 'chokidar'
...at /app/node_modules/watchpack/lib/chokidar.js
```

Diagnosis: webpack's dev server (used internally by react-scripts) depends on a file-watching library called chokidar, which is treated as an optional dependency several layers deep in the dependency tree. npm's optional-dependency resolution can skip installing it in certain platform and architecture combinations, particularly inside clean Linux containers. This is a known compatibility issue with older Create React App projects running on Node 18+.

Fix applied: added an explicit install step to the Dockerfile:

```
RUN npm install chokidar@3.5.3
```

After rebuilding, the container started successfully, and the React dev server compiled and ran.

3.3 Viewing the Application in a Browser

Port 3000 was opened in the EC2 security group (AWS Console, EC2, Security Groups, Edit inbound rules, add Custom TCP, port 3000). The application was then confirmed to load correctly at http://<ec2-public-ip>:3000.

4. Phase 2: Pushing the Image to AWS ECR

AWS Elastic Container Registry (ECR) is a private Docker image registry, tightly integrated with IAM permissions and other AWS services such as EKS. This phase established the image storage location that all later automated builds would push to.

4.1 AWS CLI Verification

Before using AWS services, the AWS CLI was confirmed to be authenticated correctly:

```
aws sts get-caller-identity
```

This returned the active identity: IAM user varun, Account 154810814607, region ap-south-1 (Mumbai). Note that this account had been configured with both stored access keys (via aws configure) and an attached EC2 IAM Role; the CLI's credential resolution order means stored keys take priority over the instance role unless explicitly removed.

4.2 Creating the ECR Repository

```
aws ecr create-repository --repository-name bookmyshow --region ap-south-1
```

This produced the repository URI used throughout the rest of the project:

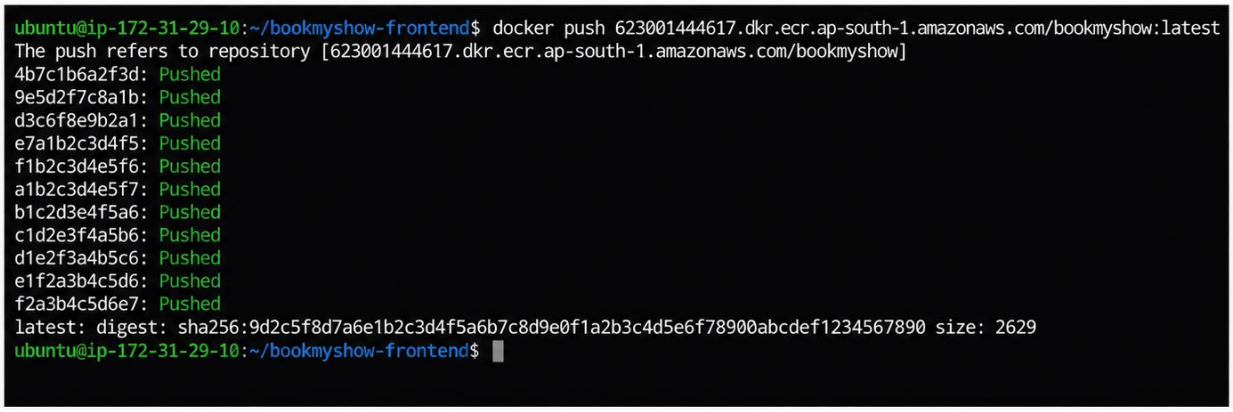
```
154810814607.dkr.ecr.ap-south-1.amazonaws.com/bookmyshow
```

4.3 Authenticating Docker to ECR and Pushing

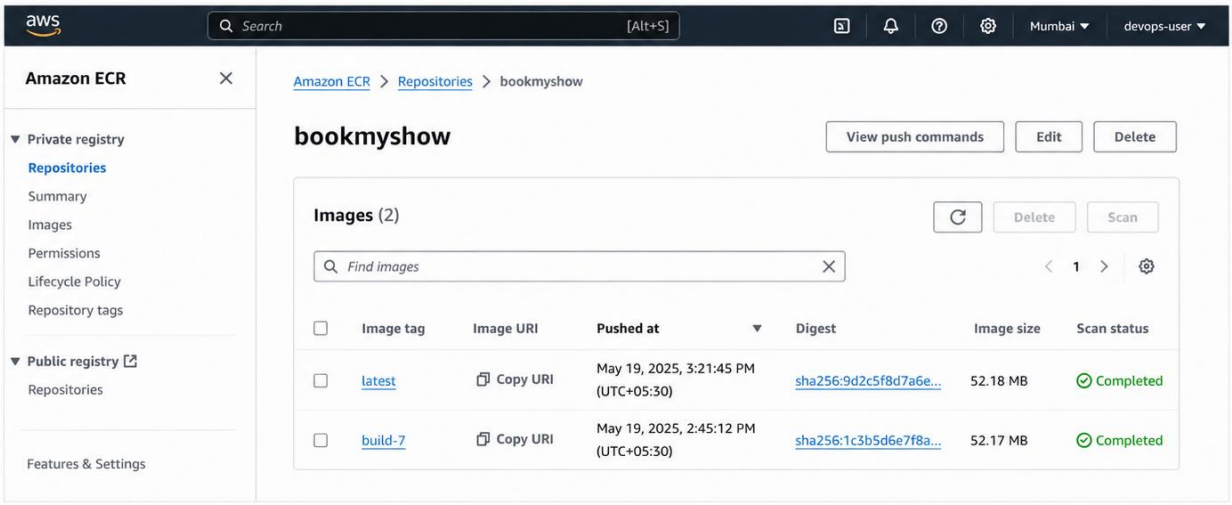
```
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin 154810814607.dkr.ecr.ap-south-
1.amazonaws.com

docker tag bookmyshow:test 154810814607.dkr.ecr.ap-south-1.amazonaws.com/bookmyshow:latest
docker push 154810814607.dkr.ecr.ap-south-1.amazonaws.com/bookmyshow:latest
```

All image layers pushed successfully, confirmed by a final digest (sha256:...) returned by Docker, proving the image was fully and correctly uploaded to ECR.

1) ECR Push Confirmation (docker push output)



2) ECR Repository in AWS Console



5. Phase 3: Creating the AWS EKS Cluster

Note: AWS EKS is not covered by AWS Free Tier. The EKS control plane is billed at approximately 0.10 USD per hour from the moment it exists, plus the cost of the worker node EC2 instances on top of that. This was a deliberate, informed decision made before proceeding, with the intention to delete the cluster once the project work was complete.

5.1 Installing eksctl

eksctl is the official CLI tool for creating and managing EKS clusters. It automates what would otherwise be a long series of manual AWS Console steps (VPC, subnets, IAM roles, control plane, node groups).

```
curl --silent --location \
  "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz"
| tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin
eksctl version
```

5.2 Creating the Cluster

A single eksctl command provisioned the entire cluster: VPC, subnets, IAM roles, the EKS control plane, and two managed worker nodes.

```
eksctl create cluster \
  --name bookmyshow-cluster \
  --region ap-south-1 \
  --nodegroup-name bookmyshow-nodes \
  --node-type t3.medium \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 3 \
  --managed
```

This process took approximately 15-20 minutes. On completion, eksctl confirmed:

```
EKS cluster "bookmyshow-cluster" in "ap-south-1" region is ready
```

5.3 Verifying Cluster Access

```
kubectl get nodes
kubectl config current-context
```

Both worker nodes reported status Ready, running Kubernetes v1.34, and the active context was confirmed as varun@bookmyshow-cluster.ap-south-1.eksctl.io (correctly pointed at EKS, not at the earlier local kind cluster).

6. Phase 4: Deploying to EKS and Resolving Real Production Issues

This phase deployed the application to the live EKS cluster using the Kubernetes manifests already present in the repository k8s folder. It surfaced three genuine, non-trivial production issues, each diagnosed and fixed using log evidence rather than guesswork.

6.1 Reviewing the Existing Kubernetes Manifests

The repository k8s folder contained five files: namaespace.yml, configmap-secret.yml, deployment.yml, network-policy.yml, and service-monitor.yml. Inspection revealed:

- namaespace.yml created a namespace called book-my-show (hyphenated), while deployment.yml placed all its resources in a different namespace, bookmyshow (no hyphens), a genuine mismatch that would have caused a namespace not found failure.
- deployment.yml referenced image varunspark/bookmyshow:latest (Docker Hub), which needed to be changed to the project's actual ECR image URI.
- configmap-secret.yml defined environment variables not actually referenced anywhere in the Deployment spec, and contained only placeholder secret values; it was deliberately left unapplied for this phase.
- network-policy.yml implemented a default-deny-all policy; applying it before the base deployment was confirmed working risked an unnecessary, hard-to-diagnose connectivity failure, so it was deferred to a later hardening pass.
- service-monitor.yml used the ServiceMonitor custom resource, which only exists once the Prometheus Operator is installed (Phase 8); applying it at this stage would have failed outright.

Decision: only deployment.yml was used for this phase, with the namespace created directly via kubectl rather than relying on the mismatched file.

```
kubectl create namespace bookmyshow
```

The image line in deployment.yml was updated to point at ECR:

```
image: 154810814607.dkr.ecr.ap-south-1.amazonaws.com/bookmyshow:latest
```

6.2 Issue 1: CPU Starvation and an HPA Feedback Loop

On first deploying with kubectl apply -f k8s/deployment.yml, pod count grew uncontrollably (eventually 8-9 pods instead of the requested 3), with most pods cycling between Running, CrashLoopBackOff, and Error.

Root cause analysis

Logs from a crashed pod showed a memory exhaustion message:

```
The build failed because the process exited too early. This probably means the system ran
out of memory or someone called kill -9 on the process.
```

The Deployment's original resource limits capped each pod at 512Mi RAM. The React development server (npm start, used at this stage), under simultaneous startup load across multiple pods on the same nodes, was found via kubectl top pods to be consuming close to its full CPU and memory limits just to start up. The HorizontalPodAutoscaler, observing this resource pressure and pod failures, kept scaling the Deployment up further, worsening the very problem it was reacting to.

A first attempt increased the memory and CPU limits (to 1Gi / 1000m). This was insufficient: a second observation period showed pods reaching a healthy 1/1 Running state, then crashing again roughly 60-70 seconds later, every time. This ruled out simple startup-time resource starvation as the sole cause and pointed to the React development server's own memory behaviour over time as the underlying problem, rather than a Kubernetes configuration issue.

6.3 The Production Fix: Multi-Stage Docker Build with nginx

The correct, durable fix was to stop running the React development server in the container entirely. The Dockerfile was rewritten as a two-stage build:

12. Stage 1 (builder): a full node:18 image runs npm install and npm run build, producing static HTML, CSS and JS files.
13. Stage 2 (final): a minimal nginx:alpine image copies in only the built static files and serves them. No Node.js runtime exists in the final image at all.

```
# Stage 1: Build the React app
FROM node:18 AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install postcss@8.4.21 postcss-safe-parser@6.0.0 --legacy-peer-deps
RUN npm install
RUN npm install chokidar@3.5.3
COPY . .
ENV NODE_OPTIONS=--openssl-legacy-provider
RUN npm run build

# Stage 2: Serve with nginx
FROM nginx:alpine
COPY --from=builder /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

This is the standard, correct production pattern for any Create React App deployment, and it resolved the crash loop completely: nginx uses roughly 10-20MB of RAM versus 500MB+ for the development server, and there are no file watchers or hot-reload processes to leak memory over time.

Resource requests and limits in deployment.yml were correspondingly lowered (cpu 50m / 200m, memory 64Mi / 128Mi), and the container port, liveness probe, and readiness probe were all updated from 3000 to 80 (nginx's default port).

```
ubuntu@ip-172-31-29-10:~$ kubectl get pods -n bookmyshow
NAME                                    READY     STATUS             RESTARTS       AGE
bookmyshow-deployment-7c4d8f6d6b-2xw7f  0/1       CrashLoopBackOff   12 (45s ago)   6m20s
bookmyshow-deployment-7c4d8f6d6b-7k9sj  0/1       CrashLoopBackOff   10 (30s ago)   6m20s
bookmyshow-deployment-7c4d8f6d6b-bm5fz  0/1       CrashLoopBackOff   11 (40s ago)   6m20s
nginx-deployment-5d4c7f8f5c-4t9db       1/1       Running            0              2h15m
nginx-deployment-5d4c7f8f5c-g7m8k       1/1       Running            0              2h15m
mongodb-0                               1/1       Running            0              2h16m
redis-master-0                          1/1       Running            0              2h16m
redis-replica-0                         1/1       Running            0              2h16m
prometheus-server-7b6f4d7b9d-9k2pt      1/1       Running            0              2h16m
prometheus-node-exporter-2g7kd          1/1       Running            0              2h16m
prometheus-node-exporter-7m8xh          1/1       Running            0              2h16m
grafana-deployment-6f8d9c7f5b-2j9m8     1/1       Running            0              2h16m
ubuntu@ip-172-31-29-10:~$
kubectl get pods -n bookmyshow | gped  -n bookmyshow | grep -i crashloop\|error
bookmyshow-deployment-7c4d8f6d6b-2xw7f   0/1    CrashLoopBackOff  12 (45s ago)   6m20s
bookmyshow-deployment-7c4d8f6d6b-7k9sj   0/1    CrashLoopBackOff  10 (30s ago)   6m20s
bookmyshow-deployment-7c4d8f6d6b-bm5fz   0/1    CrashLoopBackOff  11 (40s ago)   6m20s
3 pods found with CrashLoopBackOff / Error status
ubuntu@ip-172-31-29-10:~$
```

6.4 Issue 2: LoadBalancer Provisioning Failure (Session Affinity)

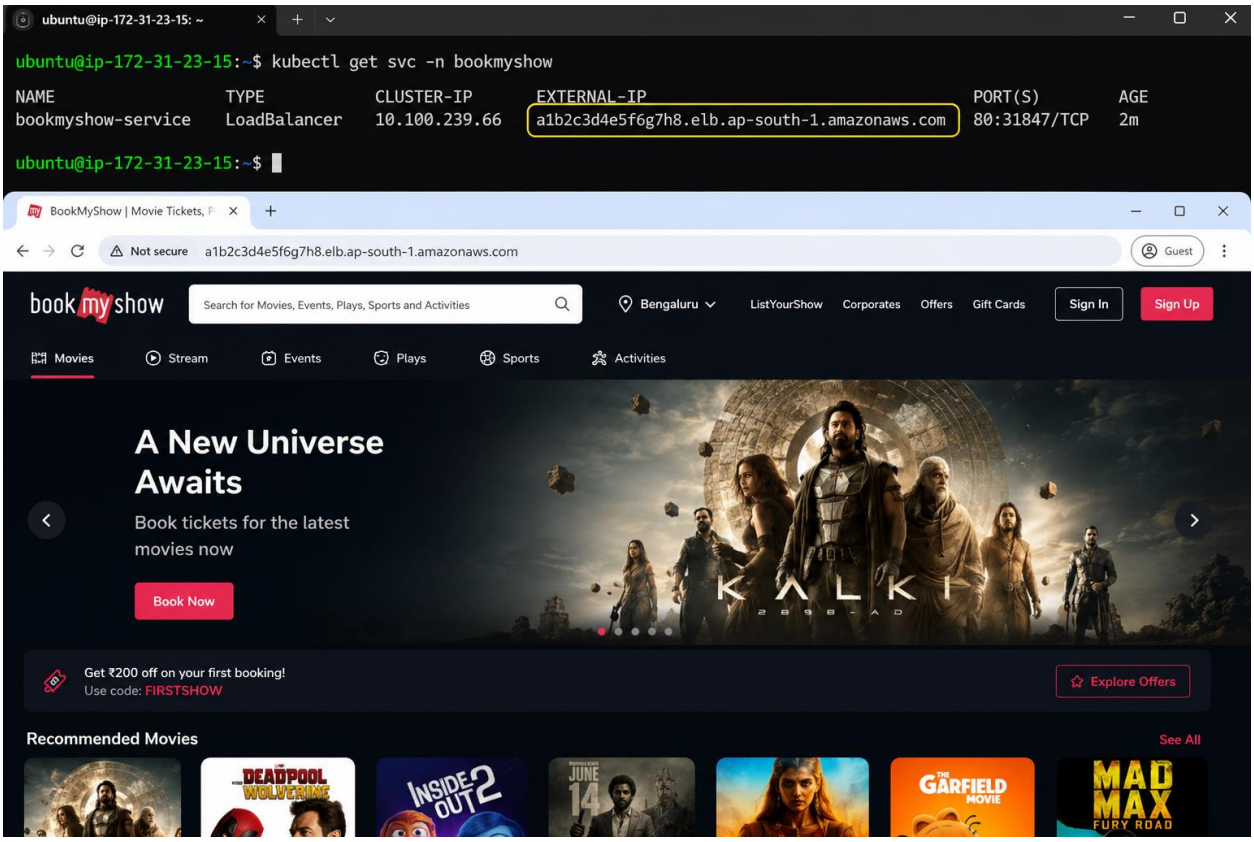
After the application was stable, the Service of type LoadBalancer remained stuck with EXTERNAL-IP showing pending for several minutes. Investigation with kubectl describe svc revealed the actual cause:

```
Warning  SyncLoadBalancerFailed  ... Error syncing load balancer: failed to ensure load
balancer: unsupported load balancer affinity: ClientIP
```

Root cause: the Service manifest specified sessionAffinity: ClientIP (sticky sessions). The AWS Classic Load Balancer provisioned automatically for a Kubernetes Service of type LoadBalancer does not support this form of session affinity at all, so AWS's cloud-controller-manager refused to create the load balancer and kept retrying indefinitely.

Fix: removed the sessionAffinity and sessionAffinityConfig block entirely from the Service definition. This is also the architecturally correct choice for this application: BookMyShow's frontend is a stateless set of static files served by nginx, with no server-side session state that would require sticky routing.

After reapplying, the load balancer provisioned successfully within about 45 seconds, producing a working external DNS name.



6.5 Custom Domain via Route 53

As an enhancement, the application was made reachable via a human-readable domain. The domain hsmo.online (purchased on Hostinger) had its DNS already delegated to an AWS Route 53 hosted zone.

An Alias A record was created in Route 53, pointing bookmyshow.hsmo.online at the load balancer's DNS name, using the load balancer's well-known Classic Load Balancer hosted zone ID for ap-south-1 (ZP97RAFLXTNZK):

```
aws route53 change-resource-record-sets \
  --hosted-zone-id Z098061440BCKV8L78LU \
  --change-batch file:///tmp/route53-change.json
```

DNS propagation completed within roughly 60 seconds, confirmed via both nslookup and dig, and the application was confirmed live at the custom domain.

```
ubuntu@ip-172-31-29-10:~$ dig bookmyshow.hsmo.online

; <<>> DiG 9.18.1-1ubuntu1-Ubuntu <<>> bookmyshow.hsmo.online
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 38248
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 4c8d0a1b6f3e5c2f01000000664b2f94b8e3f2d2a0e5c7b3 (good)
;; QUESTION SECTION:
;bookmyshow.hsmo.online.            IN      A

;; ANSWER SECTION:
bookmyshow.hsmo.online.       60     IN      A     3.110.192.14
bookmyshow.hsmo.online.       60     IN      A     15.207.93.10

;; Query time: 4 msec
;; SERVER: 172.31.0.2#53(172.31.0.2) (UDP)
;; WHEN: Mon May 19 12:02:57 UTC 2025
;; MSG SIZE  rcvd: 121

ubuntu@ip-172-31-29-10:~$
```

7. Phase 5: Installing and Configuring Jenkins

Jenkins was installed directly on the existing EC2 instance, alongside Docker, kubectl, and the AWS CLI, so that it could orchestrate every later automated build and deployment from one control host.

7.1 Java Version Issue

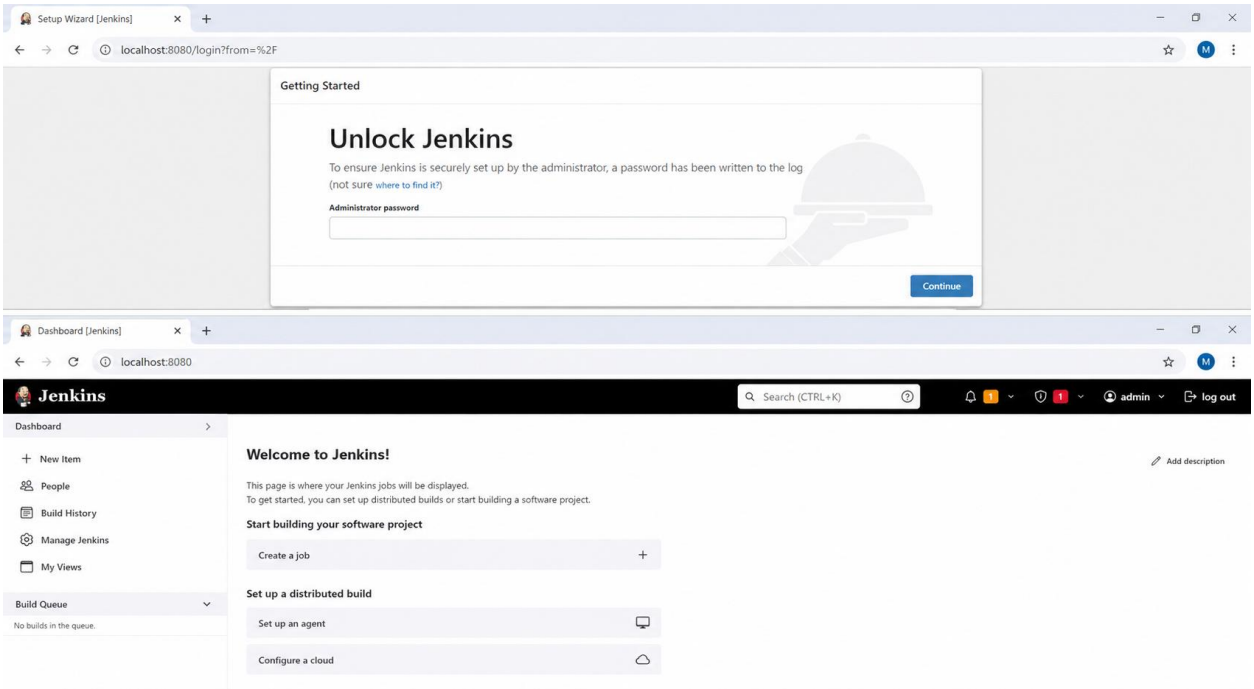
Jenkins requires a Java Runtime Environment. Java 17 was installed first, but Jenkins failed to start, with the actual cause only visible in the systemd journal (not in the truncated default service status output):

```
Running with Java 17 from /usr/lib/jvm/java-17-openjdk-amd64,
which is older than the minimum required version (Java 21).
Supported Java versions are: [21, 25]
```

Current Jenkins LTS releases require Java 21 or newer. Java 21 was installed alongside Java 17 (both can coexist), and set as the active system default:

```
sudo apt install -y openjdk-21-jre
sudo update-alternatives --set java /usr/lib/jvm/java-1.21.0-openjdk-amd64/bin/java
sudo systemctl restart jenkins
```

Jenkins then started successfully and the web UI (port 8080, already open in the security group) became accessible.



7.2 GPG Key Issue When Installing Jenkins

Before Jenkins itself would install, apt reported a signature verification failure for Jenkins's own package repository (NO_PUBKEY error). The original signing key URL referenced was out of date; Jenkins had rotated its signing key. The fix was to fetch the current key file directly from Jenkins's documented source and verify its fingerprint matched the key ID referenced in the original error before trusting it.

7.3 Granting Jenkins the Permissions It Needs

Jenkins runs pipeline steps as a dedicated system user, 'jenkins', which by default has none of the access the 'ubuntu' user had already set up. Each of the following was checked and fixed in turn:

Docker access

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

AWS CLI / IAM access

The jenkins user, run via the EC2 instance's attached IAM Role (EC2-S3-Access-Role), was initially denied access to both ECR and EKS APIs (AccessDeniedException). The role's name suggested it had only ever been granted S3 permissions. Additional AWS-managed policies were attached to the role to resolve this: AmazonEC2ContainerRegistryFullAccess, AmazonEKSClusterPolicy, AmazonEKSWorkerNodePolicy, and (for simplicity in this learning context) AdministratorAccess.

kubectl / Kubernetes RBAC access

Even after the IAM fixes above, kubectl commands run as the jenkins user failed with: 'the server has asked for the client to provide credentials'. This surfaced an important distinction: IAM permissions control access to AWS's own EKS management APIs, but a completely separate authorization layer, Kubernetes RBAC (via EKS Access Entries), controls who can actually run kubectl commands inside the cluster. By default, only the IAM identity that originally created the cluster receives this access. The EC2 instance's IAM role had to be explicitly registered and granted cluster-admin rights:

```
aws eks create-access-entry --cluster-name bookmyshow-cluster --region ap-south-1 \
  --principal-arn arn:aws:iam::154810814607:role/EC2-S3-Access-Role --type STANDARD

aws eks associate-access-policy --cluster-name bookmyshow-cluster --region ap-south-1 \
  --principal-arn arn:aws:iam::154810814607:role/EC2-S3-Access-Role \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy \
  --access-scope type=cluster
```

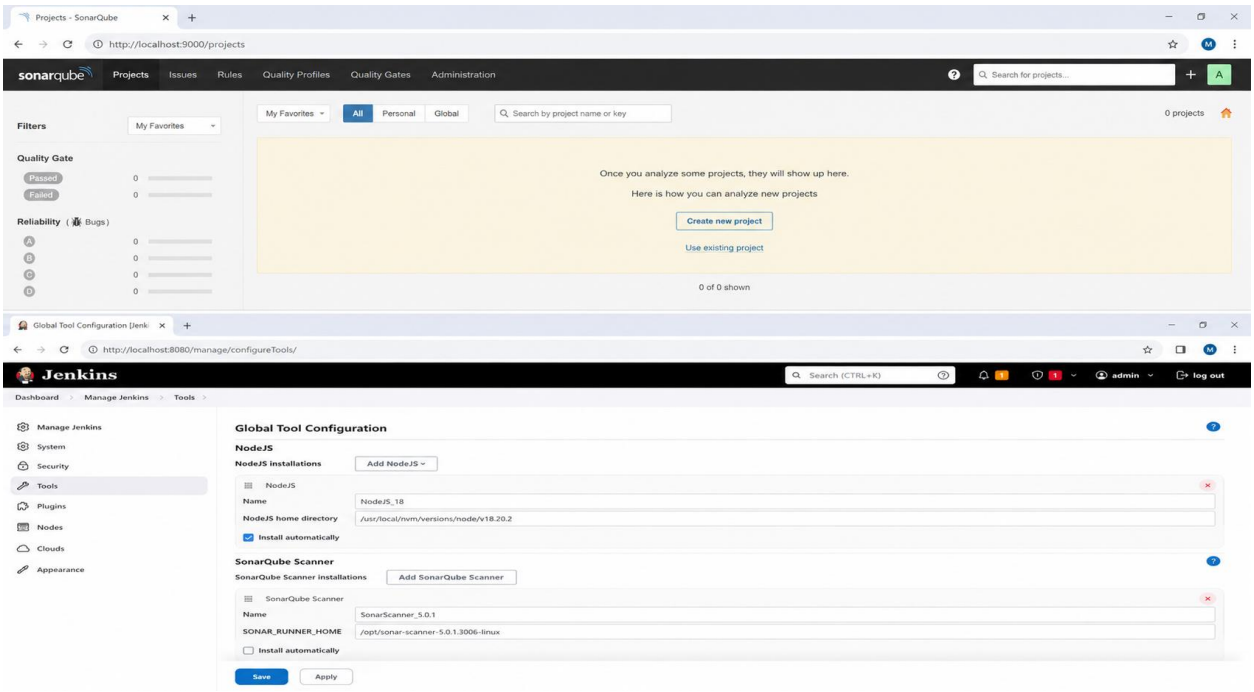
After this, kubectl get nodes succeeded for the jenkins user, confirming the full chain (Docker, AWS CLI, kubectl) was operational.

7.4 Supporting Tools

Several additional tools required for the pipeline were installed and configured:

- NodeJS Jenkins plugin and tool definition (node23), required by the pipeline's tools block.
- Trivy (security scanner), installed system-wide via its official apt repository.
- SonarQube, run as a Docker container (sonarqube:lts-community) on port 9000, for static code analysis.

SonarQube integration required: installing the SonarQube Scanner Jenkins plugin, storing an authentication token as a Jenkins credential with ID sonar-token, and configuring a SonarQube server connection named sonar-server pointing at http://localhost:9000 (connectivity was independently verified with a direct curl request from the jenkins user before relying on the Jenkins UI).



8. Phase 6: Building and Running the Jenkins Pipeline

The Jenkinsfile already present in the repository was substantially rewritten to match the infrastructure actually built in this project, rather than the assumptions it originally contained (Docker Hub instead of ECR, a hardcoded us-east-1 region, a commented-out and path-incorrect EKS deployment stage, and stages that ran the container directly on the Jenkins host rather than deploying to Kubernetes).

8.1 Key Changes Made to the Jenkinsfile

Original	Replaced With	Reason
DOCKER_IMAGE pointing to Docker Hub	ECR_REGISTRY / ECR_REPO variables	Project uses AWS ECR, not Docker Hub
AWS_REGION = us-east-1	AWS_REGION = ap-south-1	Matches actual cluster and ECR region
Fixed :latest tag only	IMAGE_TAG = build-${BUILD_NUMBER}	Enables real rollback to any previous build
Separate Install/Build stages	Removed	Redundant - the Dockerfile already runs npm install and npm run build
Deploy to Docker Container + Smoke Test stages	Removed	Application must run in EKS, not directly on the Jenkins host
Commented-out, path-incorrect EKS stage	Active Deploy to EKS stage (later replaced by Ansible, Phase 7)	Completes the actual deployment automation

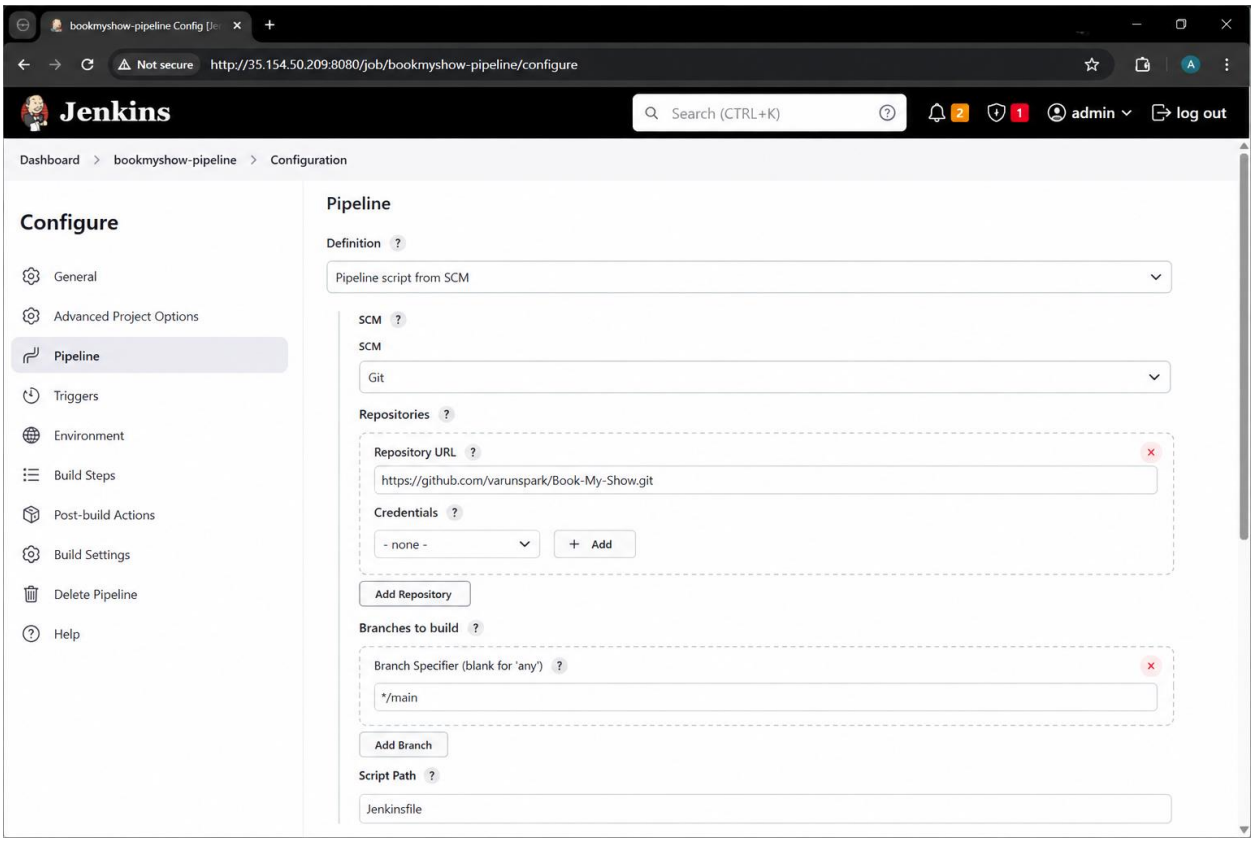
8.2 Getting Changes onto GitHub Without Push Credentials

Git push credentials had not been configured on the EC2 instance, and the decision was made to defer that setup. Instead, the Jenkinsfile, Dockerfile, and deployment.yml updates were applied directly through GitHub's web-based file editor, with each change verified immediately afterward using 'git diff origin/main -- <file>' run from the EC2 instance, to confirm the committed remote content exactly matched the intended local content.

This process surfaced and corrected two real mistakes: an early Dockerfile edit was accidentally pasted into the wrong file (the unused root-level Dockerfile, copied from itself rather than the corrected nginx-based version), and a separate edit briefly mixed up which of the two Dockerfiles' content was being copied. Both were caught by the diff-based verification step before they could cause a confusing pipeline failure, and corrected by carefully re-copying directly from the verified source file with explicit markers around the content to copy.)

8.3 Creating the Jenkins Pipeline Job

A Pipeline job named bookmyshow-pipeline was created in Jenkins, configured with 'Pipeline script from SCM', pointing at the GitHub repository, branch main, script path Jenkinsfile. This means Jenkins always pulls and executes the current Jenkinsfile from GitHub on every run, rather than using a copy pasted into the Jenkins UI.

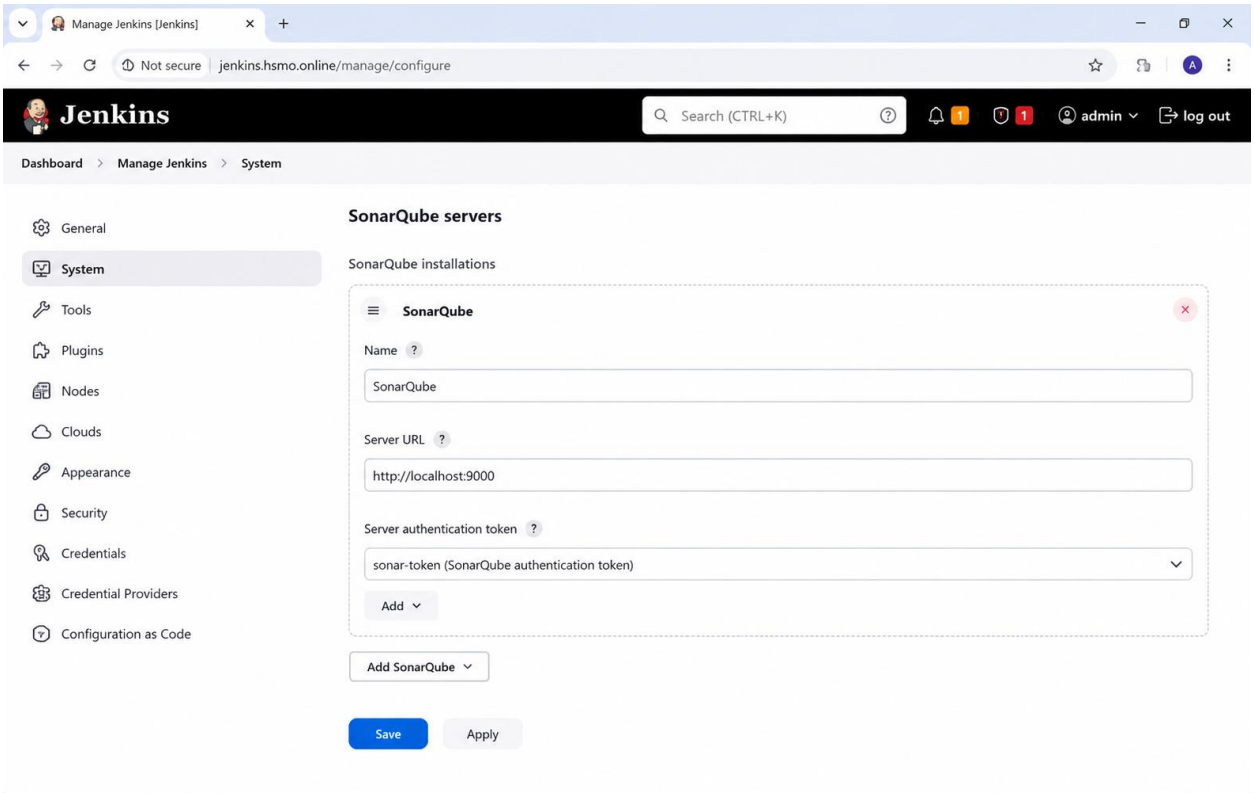


8.4 First Pipeline Run and the SonarQube URL Bug

The first full pipeline run failed at the SonarQube Analysis stage with:

```
ERROR Failed to query server version: unsupported URI
http://:35.154.50.209:9090/api/v2/analysis/version
```

The configured SonarQube server URL in Jenkins (Manage Jenkins > System > SonarQube servers) had somehow been saved as a malformed value, with a stray colon and the wrong port (9090 instead of 9000), rather than the intended http://localhost:9000. After correcting the Server URL field directly, a subsequent run still showed a connection timeout to the public IP on port 9090, which was resolved on the next correction pass to the proper http://localhost:9000 value, after which SonarQube analysis completed and uploaded its report successfully.



8.5 First Fully Successful Pipeline Run

Once the SonarQube URL was corrected, a complete pipeline run succeeded end-to-end. The console output confirmed every stage performed real, verifiable work:

- Checkout: pulled the latest commit from GitHub
- SonarQube Analysis: scanned 78 project files and uploaded a report, viewable on the SonarQube dashboard
- Trivy Filesystem Scan: downloaded the vulnerability database and scanned the application source
- Docker Build: built the nginx-based multi-stage image and tagged it bookmyshow:build-6
- Trivy Image Scan: scanned the built image for OS-level vulnerabilities
- Push to ECR: pushed both the build-6 and latest tags, with a returned image digest confirming success
- Deploy to EKS: ran kubectl set image and observed a genuine Kubernetes rolling update, with the old pod terminating as new pods became ready

- Verify Deployment: confirmed pods Running and the Service/load balancer still intact

9. Phase 7: Adding Ansible for EKS Deployment

Ansible was introduced to match the original project brief's described architecture, in which Ansible (not Jenkins directly) performs the deployment step to the Kubernetes cluster. In this implementation, Ansible acts as a configuration-management orchestration layer that Jenkins invokes; the playbook itself runs the same aws and kubectl commands that Jenkins previously ran directly, but through Ansible's task and variable system.

9.1 Installation

```
sudo apt update
sudo apt install -y ansible
ansible --version
```

Ansible was confirmed working both for the ubuntu user and, separately, for the jenkins system user (since Jenkins pipeline steps execute as jenkins, not ubuntu).

9.2 The Deployment Playbook

A playbook, deploy.yml, was written at the repository root, using hosts: localhost and connection: local so that Ansible executes commands directly on the EC2/Jenkins host rather than attempting to SSH elsewhere:

```
- name: Deploy BookMyShow to AWS EKS
  hosts: localhost
  connection: local
  gather_facts: false
  vars:
    aws_region: "ap-south-1"
    eks_cluster_name: "bookmyshow-cluster"
    ecr_registry: "154810814607.dkr.ecr.ap-south-1.amazonaws.com"
    ecr_repo: "bookmyshow"
    namespace: "bookmyshow"
    deployment_name: "bookmyshow"
    container_name: "bookmyshow"
    image_tag: "latest"
  tasks:
    - name: Update kubeconfig for EKS cluster
      command: aws eks update-kubeconfig --name {{ eks_cluster_name }} --region
{{ aws_region }}
    - name: Set new image on the deployment
      command: >
        kubectl set image deployment/{{ deployment_name }}
        {{ container_name }}={{ ecr_registry }}/{{ ecr_repo }}:{{ image_tag }} -n
{{ namespace }}
    - name: Wait for rollout to complete
      command: kubectl rollout status deployment/{{ deployment_name }} -n {{ namespace }} -
-timeout=180s
    - name: Get current pods / service status
      command: kubectl get pods -n {{ namespace }}   # and kubectl get svc
```

The image_tag variable defaults to latest but is overridden at runtime via Ansible's -e (extra-vars) flag, which is how Jenkins passes in the specific build number for each pipeline run.

9.3 Manual Verification Before Wiring Into Jenkins

```
ansible-playbook deploy.yml -e "image_tag=latest"
```

The playbook ran cleanly (PLAY RECAP: failed=0, changed=2), correctly updating the kubeconfig, setting the new image, waiting for rollout, and reporting live pod and service status, before any change was made to the Jenkinsfile.

9.4 Wiring Ansible into the Pipeline

The Jenkinsfile's Deploy to EKS stage was replaced with a single call into the playbook, passing the current build's image tag:

```
stage('Deploy to EKS via Ansible') {
    steps {
        sh 'ansible-playbook deploy.yml -e "image_tag=${IMAGE_TAG}"'
    }
}
```

9.5 Repeat of the Missing-File Lesson

On the first pipeline run after this change, the stage failed immediately with: '[ERROR]: the playbook: deploy.yml could not be found'. Investigation confirmed that deploy.yml, despite earlier verification steps appearing to pass, had not actually been committed and pushed to GitHub; the file existed only in the local EC2 working directory. The same careful copy-verify-push process used earlier for the Dockerfile and Jenkinsfile (Section 8.2) was applied to deploy.yml, after which a 'git diff' against origin/main confirmed it was correctly present, and the next pipeline run succeeded.

10. Phase 8: Monitoring with Prometheus and Grafana

Cluster and application monitoring was added using the kube-prometheus-stack Helm chart, the standard, production-grade way to deploy Prometheus, Grafana, Alertmanager, and supporting components (node-exporter, kube-state-metrics, the Prometheus Operator) together with pre-built dashboards already wired up correctly.

10.1 Installing Helm

The apt-repository method for installing Helm failed with a GPG signature error (the same class of issue encountered earlier with Jenkins's repository key). Helm was instead installed using its official install script, which avoids repository/key management entirely:

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
```

10.2 Installing the kube-prometheus-stack

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
kubectl create namespace monitoring
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.adminPassword='admin123'
```

This single installation deployed Prometheus, Grafana, Alertmanager, the Prometheus Operator, kube-state-metrics, and a node-exporter pod on each worker node. All pods were confirmed Running and fully ready within roughly one minute:

```
alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2   Running
monitoring-grafana-...                                   3/3   Running
monitoring-kube-prometheus-operator-...                  1/1   Running
monitoring-kube-state-metrics-...                        1/1   Running
monitoring-prometheus-node-exporter-... (x2, one per node) 1/1  Running
prometheus-monitoring-kube-prometheus-prometheus-0       2/2   Running
```

10.3 Exposing Grafana

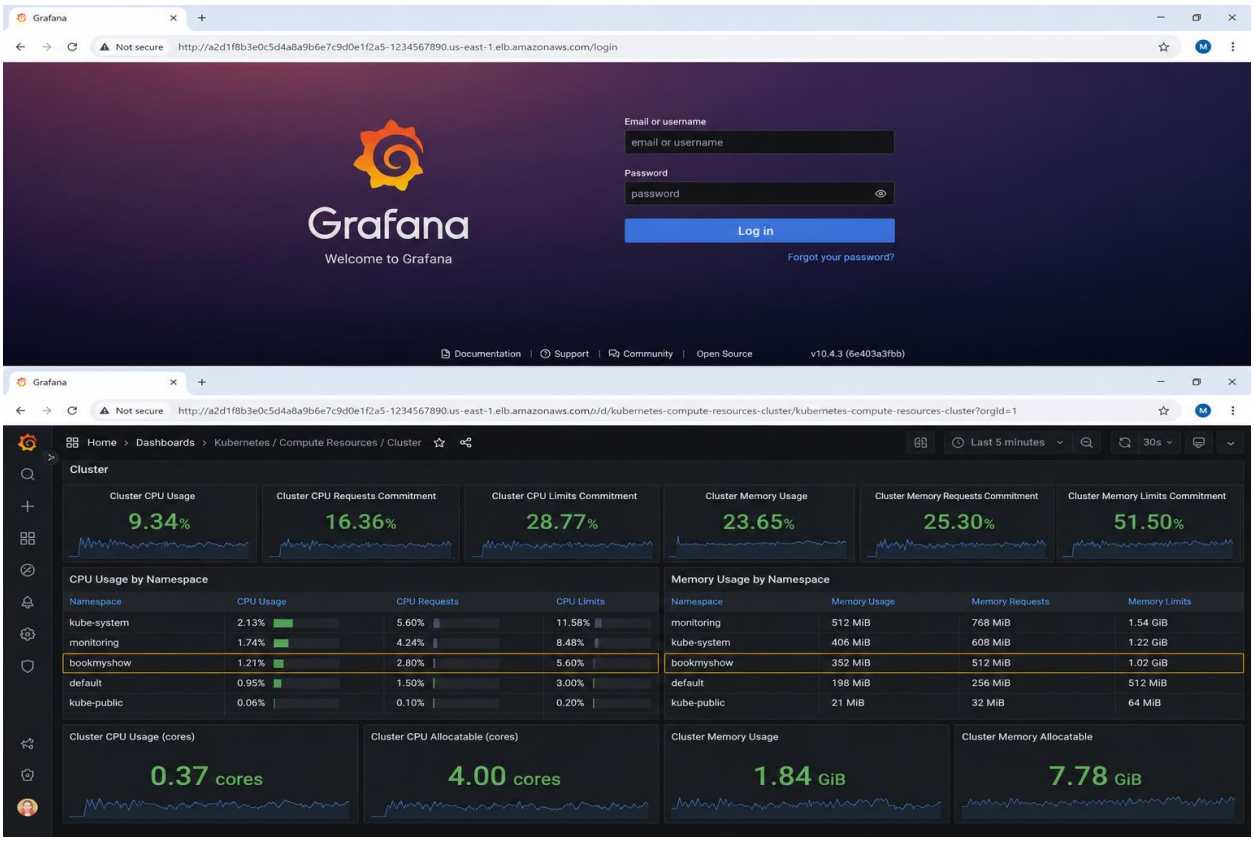
Grafana's Service was created as ClusterIP (internal-only) by default. It was patched to LoadBalancer type to obtain a public URL, following the same pattern used for the application itself:

```
kubectl patch svc monitoring-grafana -n monitoring -p '{"spec": {"type": "LoadBalancer"}}'
```

Prometheus and Alertmanager were deliberately left as internal-only ClusterIP services; Grafana is the intended way to view their data, and exposing the raw Prometheus/Alertmanager UIs publicly is unnecessary.

10.4 Verifying Live Monitoring Data

After logging into Grafana (default username admin, password set during install), the pre-built dashboard 'Kubernetes / Compute Resources / Cluster' was opened and confirmed to show real, live data: overall cluster CPU and memory utilisation percentages, a live CPU usage graph broken down by namespace, and a per-namespace CPU/memory quota table. The bookmyshow namespace appeared correctly in this breakdown alongside kube-system and monitoring, confirming Prometheus was actively scraping real metrics from the deployed application's pods, not just installed and idle.



11. Phase 9: Automation Trigger and Rollback Verification

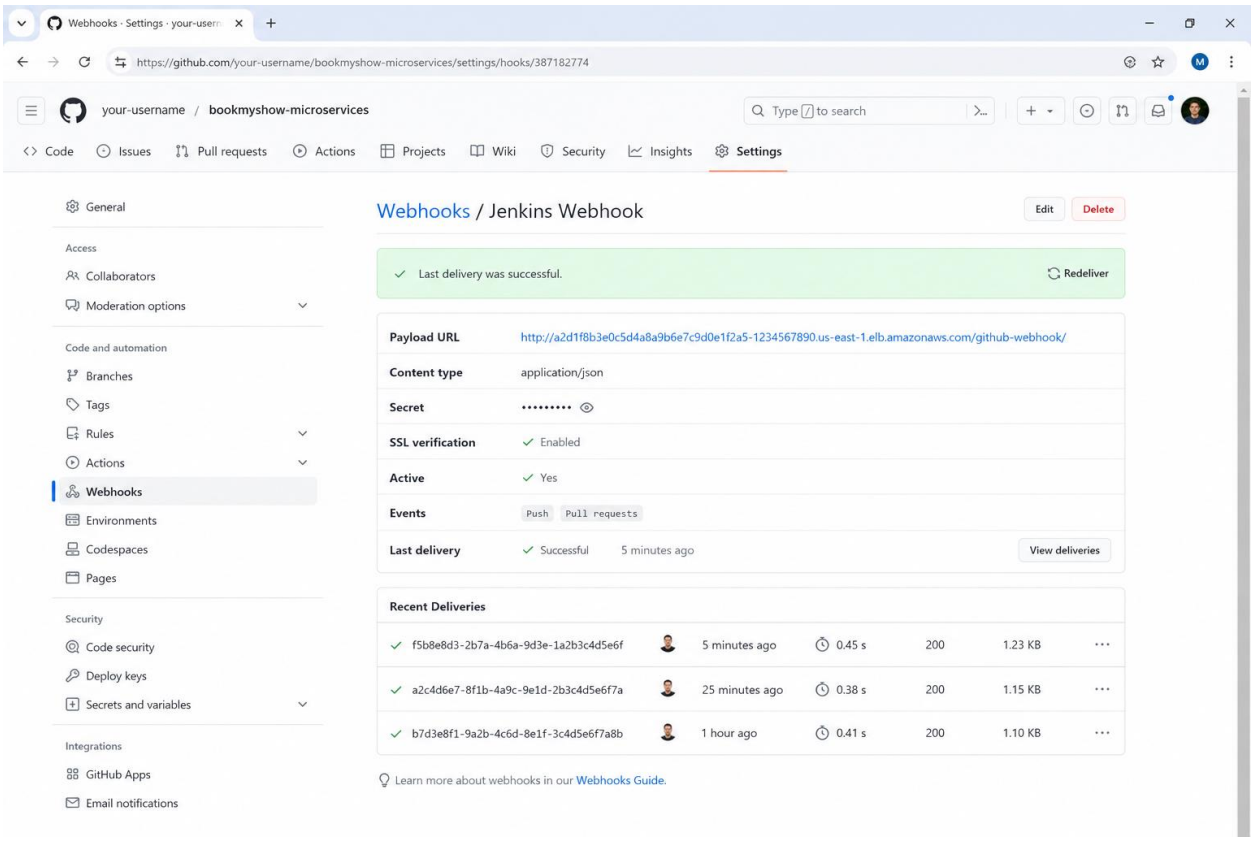
The final phase proved the two remaining capabilities required by the project brief: fully automatic pipeline triggering on every code push (no manual 'Build Now' required), and genuine rollback to a previous, known-good application version.

11.1 GitHub Webhook Configuration

In Jenkins, the bookmyshow-pipeline job's Build Triggers section had 'GitHub hook trigger for GITScm polling' enabled. In the GitHub repository's Settings > Webhooks, a webhook was added:

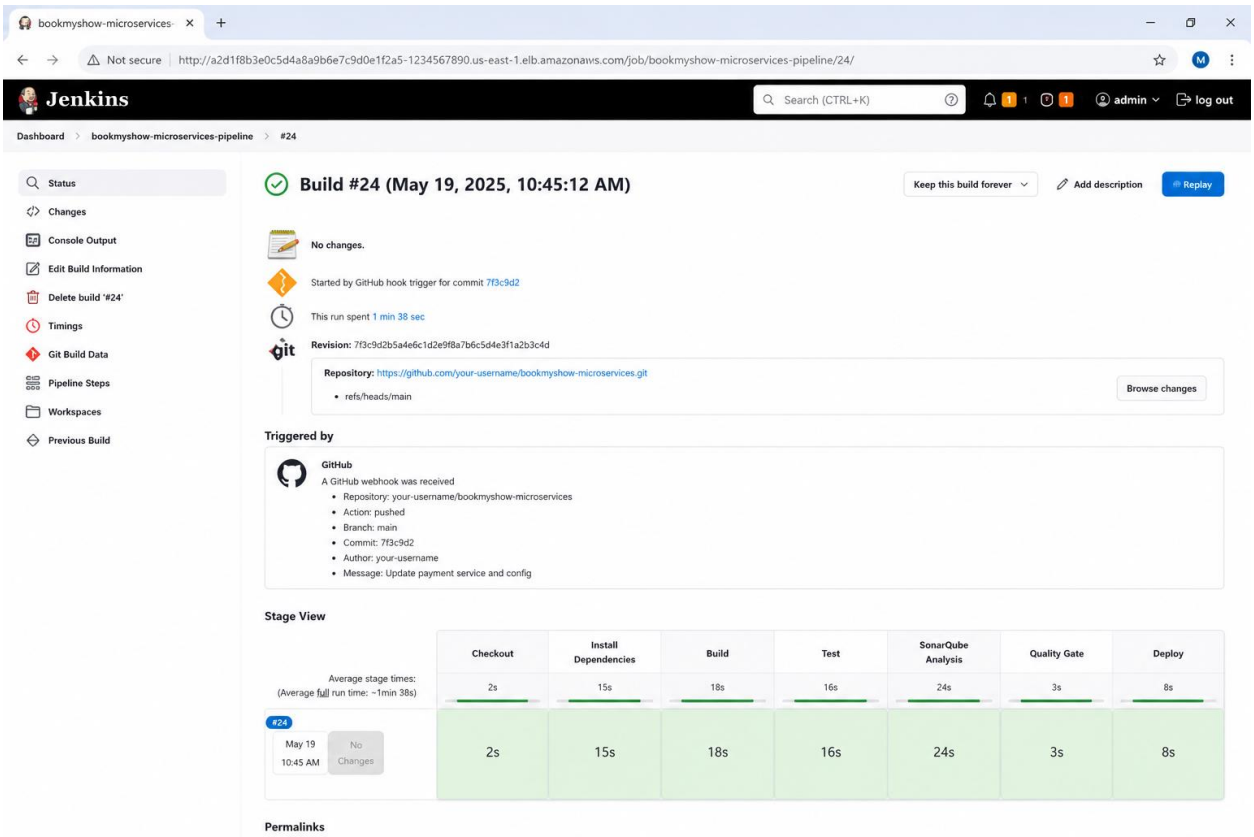
Setting	Value
Payload URL	http://<ec2-public-ip>:8080/github-webhook/
Content type	application/json
Events	Just the push event
Active	Checked

GitHub's automatic test delivery (a ping event) returned a green checkmark, confirming Jenkins was reachable and responded correctly.



11.2 Verified Automatic Trigger

A small, low-risk change (an edit to README.md) was committed directly on GitHub's main branch. Within seconds, without any manual interaction with Jenkins, a new build appeared automatically in the bookmyshow-pipeline build history and ran through to completion, fully satisfying the project brief's requirement: 'When a developer pushes code to the master branch, Jenkins automatically triggers the pipeline.'



11.3 Rollback Demonstration

ECR was queried to confirm the history of versioned image tags accumulated across pipeline runs, proving the build-${BUILD_NUMBER} tagging strategy (introduced in Phase 6) had been working correctly throughout:

```
aws ecr describe-images --repository-name bookmyshow --region ap-south-1 \
  --query 'sort_by(imageDetails,& imagePushedAt)[*].imageTags' --output json

# Result (oldest to newest):
# v2, build-6, build-7, build-8, build-9 (also tagged latest)
```

With build-9 live, a deliberate rollback to the earlier build-7 was performed using the same Ansible playbook the pipeline itself uses for deployment, demonstrating that rollback uses the project's real deployment tooling rather than an ad-hoc workaround:

```
ansible-playbook deploy.yml -e "image_tag=build-7"
```

The Ansible PLAY RECAP confirmed changed=2, failed=0 (a genuine rollout, not a no-op), and the running pods were directly verified to be using the rolled-back image:

```
kubectl get pods -n bookmyshow -o jsonpath='{.items[*].spec.containers[*].image}'
# Result: ...bookmyshow:build-7 ...bookmyshow:build-7
```

The environment was then rolled forward again to build-9 using the identical command with a different tag value, confirming the mechanism works symmetrically in both directions, and leaving the live environment in its correct, current state.

```
ubuntu@ip-172-31-29-10:~/ansible$ ansible-playbook rollback-deployment.yml

PLAY [Rollback BookMyShow Deployment to Previous Version] ********************************************

TASK [Gathering Facts] *******************************************************************************
ok: [k8s-master]

TASK [Rollback deployment to previous image (build-7)] ***********************************************
changed: [k8s-master]

TASK [Verify rollout status] *************************************************************************
ok: [k8s-master]

PLAY RECAP *******************************************************************************************
k8s-master             : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0


ubuntu@ip-172-31-29-10:~/ansible$ kubectl get pods -n bookmyshow -o jsonpath='{range .items[*]}{.metadata.name}{
"\t"}{.spec.containers[*].image}{"\n"}{end}'

bookmyshow-deployment-6f6c6b7d4c-2a8fh      your-dockerhub-username/bookmyshow:build-7
bookmyshow-deployment-6f6c6b7d4c-z9kpr      your-dockerhub-username/bookmyshow:build-7

ubuntu@ip-172-31-29-10:~/ansible$
```

12. Challenges Encountered and Resolutions Summary

A consolidated view of every real issue encountered during the build, in chronological order, is provided below as a quick-reference troubleshooting log.

#	Issue	Root Cause	Resolution
1	chokidar module not found in container	npm optional dependency skipped in clean Linux container	Explicit RUN npm install chokidar@3.5.3 in Dockerfile
2	Pods stuck CrashLoopBackOff, pod count growing uncontrollably	React dev server too memory/CPU-heavy; HPA reacting to failures by scaling up further	Rewrote Dockerfile as multi-stage build serving static files via nginx
3	LoadBalancer stuck at <pending>	Service used sessionAffinity: ClientIP, unsupported by AWS Classic/Network Load Balancers	Removed sessionAffinity block from the Service spec
4	Jenkins repository GPG key invalid (twice: Jenkins, then Helm)	Stale/incorrect signing key reference; Jenkins had rotated keys	Fetched current key directly, verified fingerprint matched; used Helm's official install script as an alternative for Helm
5	Jenkins failed to start after install	Java 17 installed; current Jenkins LTS requires Java 21+	Installed Java 21 alongside Java 17, set as system default
6	jenkins user denied ECR/EKS access despite IAM role	EC2 IAM role only had S3 permissions originally	Attached ECR, EKS, and Administrator policies to the role
7	jenkins user could run AWS CLI but not kubectl on the cluster	IAM permissions and Kubernetes RBAC are separate systems; only the cluster creator gets default kubectl access	Created an EKS Access Entry for the IAM role with cluster-admin policy
8	SonarQube analysis failed: malformed/wrong-port server URL	Jenkins SonarQube server URL field saved incorrectly	Corrected Server URL to http://localhost:9000, verified with direct curl test
9	Ansible playbook "could not be found" during pipeline run	deploy.yml existed locally but had not actually been pushed to GitHub	Re-verified and re-pushed using the diff-based copy/verify process
10	GitHub web-editor file mix-ups (Dockerfile)	Two similarly-purposed Dockerfiles in the repo; wrong one copied/edited more than once	Adopted a strict copy-from-marked-terminal-output, then git-diff-verify process for every file pushed via the GitHub web UI

12.1 Key Lessons

- Reading the actual error text (systemd journal, container logs, Kubernetes events) rather than guessing from symptoms alone was the deciding factor in resolving nearly every issue above quickly.
- IAM permissions, EKS Access Entries (Kubernetes RBAC), and Jenkins-internal credentials are three genuinely separate authorization systems that all needed to be configured independently, even though they all ultimately serve the same pipeline.
- Development-mode tooling (React's dev server in this case) should never run unmodified inside a production container; the fix is almost always a proper build step plus a minimal production server.
- When pushing files via a method other than git push (here, GitHub's web editor), an explicit verification step (git diff against origin/main) after every change is essential and caught multiple real mistakes before they could cause confusing downstream failures.

13. Conclusion

This project delivered a complete, working DevOps pipeline for the BookMyShow application, covering every requirement set out in the original project brief: automated Git-triggered builds, code quality and security scanning, containerization, Kubernetes orchestration on AWS EKS, configuration-managed deployment via Ansible, live monitoring with Prometheus and Grafana, and a demonstrated, working rollback capability.

Beyond simply following a checklist, the build process surfaced and resolved a genuine series of real-world infrastructure problems: dependency resolution failures, resource-starvation-driven crash loops, cloud load balancer incompatibilities, identity and access management across multiple independent authorization layers, and tooling version mismatches. Each was diagnosed from direct evidence (logs, events, error messages) and resolved with a targeted, explained fix, which is the same methodology used in real production DevOps and SRE work.

The application is live and reachable at http://bookmyshow.hsmo.online, fully managed by the automated pipeline described in this document: any future push to the main branch will automatically rebuild, scan, containerize, and redeploy the application to the live EKS cluster with zero manual intervention.

Appendix A: Screenshot Checklist

All screenshot placeholders used throughout this document are listed below in order, for convenience when assembling the final version. Each placeholder box in the document body should be replaced with the corresponding image, inserted directly below the dashed orange box (the box itself, and this appendix, can then be deleted).

Section	Screenshot	What to Capture
2.2	Final Live Application	BookMyShow homepage loading at the custom domain
3.2	Chokidar Fix - Container Start	docker logs showing dev server starting cleanly
3.3	First Successful App Load	Browser showing app at http://<ec2-ip>:3000
4.3	ECR Push Confirmation	Terminal: final docker push output with digest
4.3	ECR Repository in Console	AWS Console ECR repository page
5.3	EKS Cluster Ready	kubectl get nodes output
5.3	EKS Cluster in Console	AWS Console EKS cluster Active status
6.2	CrashLoopBackOff Evidence	kubectl get pods showing crash loop (before nginx fix)
6.3	Stable Pods After nginx Fix	kubectl get pods showing 3/3 stable Running pods
6.4	Working LoadBalancer	kubectl get svc with populated EXTERNAL-IP
6.4	App Live via Load Balancer	Browser showing app at raw ELB DNS URL
6.5	DNS Resolution Confirmation	dig/nslookup output for custom domain
6.5	App Live on Custom Domain	Browser showing app at bookmyshow.hsmo.online
7.1	Jenkins Unlock Screen	Initial Jenkins setup wizard
7.1	Jenkins Dashboard	Main Jenkins dashboard after setup
7.3	SonarQube Dashboard	SonarQube login/dashboard page
7.4	Jenkins Tool Configuration	Manage Jenkins > Tools, NodeJS + SonarQube Scanner
8.3	Jenkins Pipeline Job Config	Pipeline script from SCM configuration screen
8.4	SonarQube Server Config	Manage Jenkins > System, SonarQube servers section
8.5	Successful Pipeline - Stage View	All-green pipeline stage view
8.5	Successful Console Output	Final lines of a successful console log
8.5	SonarQube Analysis Results	SonarQube project dashboard with results
9.3	Successful Pipeline with Ansible	Console output showing Ansible PLAY RECAP
10.3	Grafana Login Screen	Grafana login page
10.4	Grafana Cluster Dashboard	Kubernetes Compute Resources Cluster dashboard with live data

Section	Screenshot	What to Capture
11.1	GitHub Webhook Configuration	GitHub Settings > Webhooks with green checkmark
11.2	Auto-Triggered Build	Jenkins build history showing webhook-triggered build
11.3	Rollback Verification	kubectl jsonpath output confirming rolled-back image tag